CHIMERA — project reference

Delay-Aware Predictive Coupled Autoscaling for Kubernetes · BCSE303P Operating Systems Laboratory · Adithyan C S S (24BCE2820), Vishal B S (24BCE2475), Madhan A (24BCE0159)

Where the project lives

WHAT	PATH
Project root	`C:\Users\adith\.gemini\antigravity\scratch\chimera_project`
Controller	`src\core\chimera_controller.py`
Simulator	`src\core\chimera_simulator.py`
Baselines + harness	`src\scripts\run_full_evaluation.py`
KOSMOS baseline	`src\core\kosmos_simulator.py`
Tests	`tests\test_chimera.py`
Datasets	`dataset\workloads\w_0001.json … w_0005.json`
Verified results	`results\results_FINAL_verified.csv`
Kubernetes operator	`k8s\crd.yaml, controller.yaml, manager.py, Dockerfile`

How to run it

PowerShell, from the project root. Python 3.10 or newer.

```
# 1 — install dependencies (once)
pip install -r requirements.txt

# 2 — unit tests  (~1 s, expect "Ran 6 tests ... OK")
$env:PYTHONPATH="."; python tests\test_chimera.py

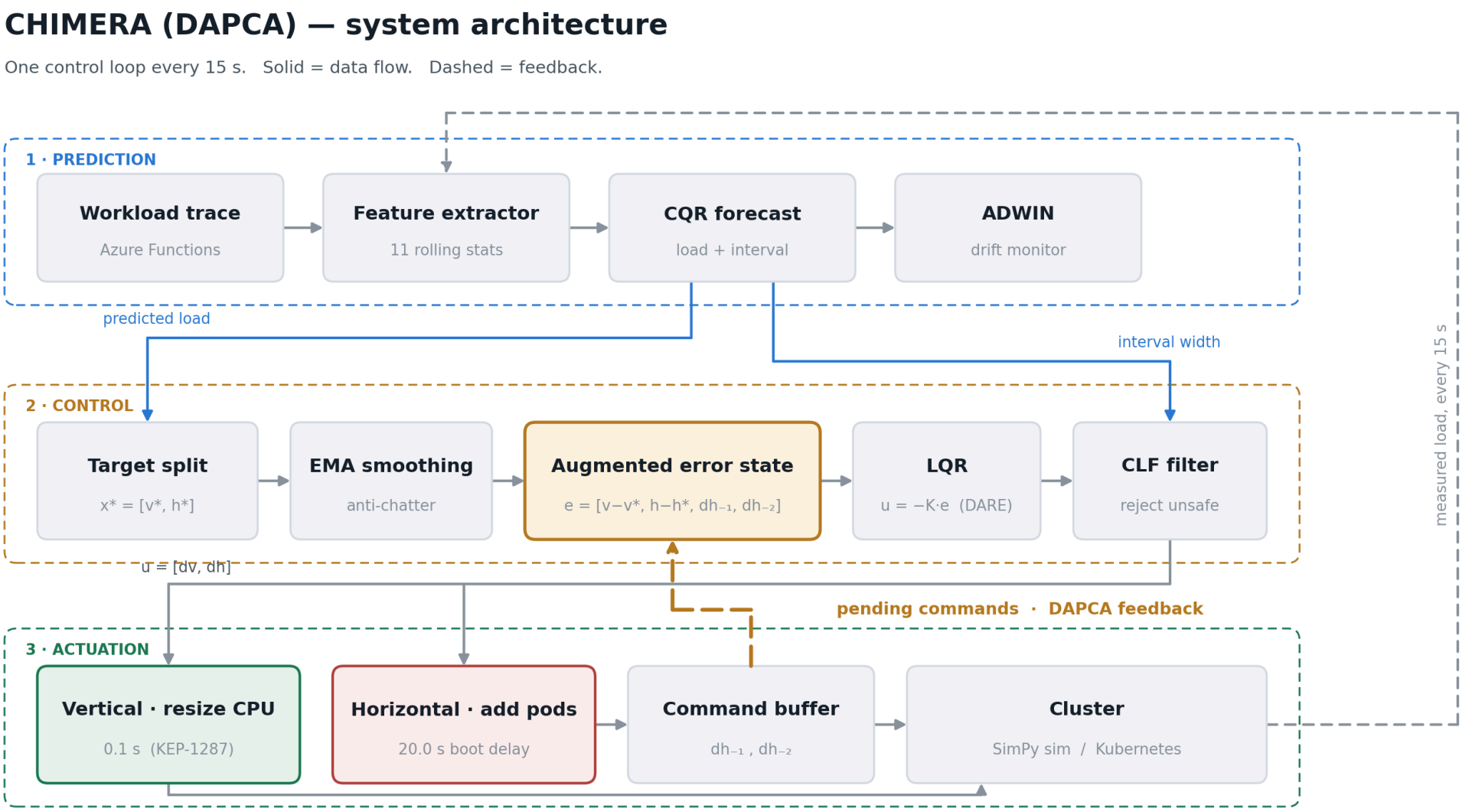
# 3 — full evaluation: 27 configs x 9 workloads x 3 seeds
#     writes results\results.csv and two more
$env:PYTHONPATH="."
python src\scripts\run_full_evaluation.py

# 4 — single-trace demo: CHIMERA vs HPA  (~2 s)
$env:PYTHONPATH="."; python demo_run.py

# 5 — deploy the operator to a local cluster
kubectl apply -f k8s\crd.yaml
kubectl apply -f k8s\controller.yaml
```

Note: step 3 overwrites the results files and takes several minutes. The verified run used for every number in the report is preserved separately as `results\results_FINAL_verified.csv`.

System architecture



Three pipelines run as one loop every 15 seconds. The amber path is the project's distinguishing feature: scaling commands already issued but not yet in effect are fed back into the controller's state, so it does not re-issue capacity that is already on its way. The blue paths carry the forecast and its confidence interval — the interval widens the safety filter's tolerance when the forecast is uncertain.

Datasets used

Nine workloads. Five are real traces from the public Microsoft Azure Functions 2019 dataset; four are synthetic scenarios written to stress specific failure modes. Every controller is evaluated on all nine, with three random seeds each.

WORKLOAD	SOURCE	PATTERN	MEAN	PEAK	WHY IT IS IN THE SET
real_kswd_w1	Azure Functions 2019 (`w_0001`)	Spike	84	120	Sudden bursts — tests reaction to unforecast load
real_kswd_w2	Azure Functions 2019 (`w_0002`)	Periodic	846	1035	Regular daily rhythm — tests the forecaster
real_kswd_w3	Azure Functions 2019 (`w_0003`)	Spike	83	120	Second spike trace, uncorrelated with w1
real_kswd_w4	Azure Functions 2019 (`w_0004`)	Ramp	303	450	Steady growth — tests tracking without overshoot
real_kswd_w5	Azure Functions 2019 (`w_0005`)	Stationary noisy	734	1320	Noisy but level — tests resistance to chattering
synthetic_coupled_oscillation	Authored	Alternating	—	—	Forces the two actuators into conflict on purpose
synthetic_drift	Authored	Distribution shift	—	—	Triggers the ADWIN drift detector
synthetic_extreme_overload	Authored	Saturation	—	—	Drives the cluster past its resource ceiling
synthetic_stress_uncertainty	Authored	Unpredictable	—	—	Makes the forecast interval very wide

Protocol. Each trace is 100 minutes of requests-per-minute, scaled by 15. Only minutes 70–100 are scored; minutes 10–50 train the forecaster and 50–70 calibrate it, so no test data reaches the model. 27 configurations × 9 workloads × 3 seeds = **720 verified runs**.

Known limitation, stated openly. The five real traces are 100-minute *excerpts* of 14-day traces. The full Azure Functions 2019 dataset holds over 10,000 workloads, and `dataset\tools\azure_to_k8s_converter.py` exists to convert it. Running on the full dataset is the next milestone; until then, generalisation claims are limited to these five.

Comparison with existing systems

Scaling behaviour of each system, and its measured result on the five real traces where we ran it. Lower is better on every number shown.

SYSTEM	HORIZ.	VERT.	ONE COORDINATED LAW	DELAY-AWARE	UNCERTAINTY MODEL	SLO VIOL.	P99 (S)	POD-MIN	ACTIONS
Kubernetes HPA	✓	—	n/a	—	none	17.15 %	10.16	215	10
Kubernetes VPA	—	✓	n/a	—	none	16.18 %	1.98	150	6
HPA + VPA together	✓	✓	no — independent loops	—	none	17.15 %	10.16	215	10
AAPA (base paper)	✓	—	n/a	—	classifier confidence	8.67 %	8.74	272	5
MagicScaler-inspired	✓	—	n/a	—	peak window	73.74 %	34.83	90	4
KOSMOS (ICSOC 2021)	✓	✓	partial — separate laws	—	none	13.55 %	21.58	97	261
CHIMERA (ours)	✓	✓	yes — single LQR law	yes — state augmentation	conformal interval	9.74 %	3.57	147	13

Related work not run in our simulator

SYSTEM	WHAT IT DOES	HOW CHIMERA DIFFERS
GKE Multidimensional Pod Autoscaler	Google product running HPA and VPA together	It avoids conflict by giving each loop a different metric (CPU vs memory). It never coordinates both on the same resource.
ADAPT (arXiv 2605.15788, 2026)	Delay-aware autoscaling via model predictive control	Horizontal only, and solves an optimisation at every step. CHIMERA handles both axes with a precomputed gain.
OptScaler (VLDB 2024), BACC (arXiv 2606.20575, 2026)	Conformal prediction for autoscaling	Both horizontal only. CHIMERA feeds the conformal interval into a stability constraint rather than a replica budget.
IBM US 8,756,609 B2 · CenturyLink US 9,329,904 B2	Patents on two-dimensional scaling	Both are rule-based threshold selectors that pick one axis or the other. CHIMERA drives both at once from one control law.

How to read the results columns. No system wins on everything, and quoting a violation rate alone is misleading — any scaler can lower it by holding more spare capacity. AAPA beats CHIMERA on violations while spending 85 % more resource; KOSMOS is cheaper but 6× worse on tail latency and makes 20× more scaling changes. CHIMERA's claim is the balance: competitive on violations, best-in-set on tail latency, and by far the least disruptive to a running cluster.